

Next-Token Prediction

Tokens, embeddings and neural language models

Prof. Nipun Batra

ES 667 · Deep Learning · IIT Gandhinagar

What is sequence modeling?

Where we are

The vision lectures took a **fixed-shape** input — an image tensor:

$$X \in \mathbb{R}^{H \times W \times C}$$

Every example had the **same** shape; a CNN mapped it to one label.

This lecture changes the **input type**. We move from fixed tensors to **sequences** of variable length — the first step toward language models.

Images are fixed; sequences are ordered

An image is a fixed grid. A **sequence** is an ordered list of tokens $x_1, x_2, \dots, x_T$:

Sequence	Tokens x_t
text	characters, subwords, or words
audio	waveform samples / frames
sensors	readings over time
events / logs	user clicks, actions
DNA	bases A, C, G, T

length T **varies** from example to example — a fixed-size vector no longer fits

Order carries meaning

The **same** tokens in a **different order** mean different things:

“dog bites man” $\neq$ “man bites dog”

A model f over sequences must **preserve order**. Treating $x_{1:T}$ as an unordered bag throws away exactly the structure that makes it a sequence.

The sequence tasks v

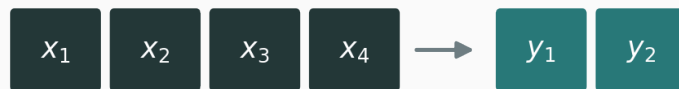
A handful of task shapes cover most of sequence modeling:

classification $x_{1:T} \rightarrow y$



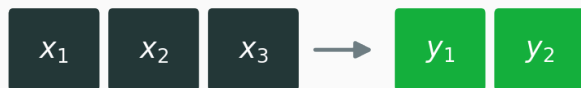
single label

token classification $\rightarrow y_{1:T}$



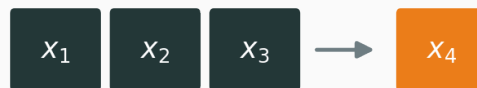
one label / token

seq2seq $\rightarrow y_{1:U}$



new length U

next-token $x_{1:t} \rightarrow x_{t+1}$



predict the next

The sequence tasks, precisely

Task	Shape	Example
classification	$x_{1:T} \rightarrow y$	sentiment of a review
token classification	$x_{1:T} \rightarrow y_{1:T}$	tag each word (POS)
seq2seq	$x_{1:T} \rightarrow y_{1:U}$	translate a sentence
next-token	$x_{1:t} \rightarrow x_{t+1}$	predict what comes next

this lecture: **next-token prediction** — the task behind language models

Our first task

Given a prefix, predict what comes next:

“deep learning is” $\xrightarrow{?}$???

The model outputs a **probability distribution** over the whole vocabulary V :

$$p_{\theta}(x_{t+1} = v \mid x_{1:t}) \quad \text{for every } v \in V$$

not a single guess — a **score for every possible next token**

Running example: makemore names

Our anchor all lecture: a **character-level name generator**.

- predict the next **character** of a name: emma, olivia, ava, noah;
- tiny vocabulary — 26 letters plus one special token;
- embeddings small enough to **see**, training fast enough to **play with**.

This is Karpathy's **makemore** (part 2), which is exactly the neural language model of **Bengio et al. 2003, "A Neural Probabilistic Language Model"**. We build up to that model in this lecture.

The probability of a sequence

A model assigns probability to sequences

A language model is really a **probability distribution** over sequences:

$$p_{\theta}(x_1, x_2, \dots, x_T)$$

How can one model score a sequence of **arbitrary length**? Factor it into pieces.

The chain rule of probability

Any joint distribution factorizes into a product of conditionals:

$$p(x_1, \dots, x_T) = p(x_1) p(x_2 \mid x_1) p(x_3 \mid x_1, x_2) \dots p(x_T \mid x_{1:T-1})$$

Compactly, with $x_{<t} = x_{1:t-1}$:

$$p(x_{1:T}) = \prod_{t=1}^T p(x_t \mid x_{<t})$$

exact — no approximation. This holds for **any** sequence distribution.

Why next-token is enough

Read the chain rule the other way around:

If we can model **every** conditional $p(x_t \mid x_{<t})$, we have modelled the **whole** sequence. Each factor is a next-token prediction.

language modeling = repeated classification

each step: a V -way classification of “which token comes next?”

The training objective falls out

Take the log of the chain-rule product — products become sums:

$$\log p(x_{1:T}) = \sum_{t=1}^T \log p(x_t \mid x_{<t})$$

Maximizing likelihood = minimizing the **negative log-likelihood**:

$$\mathcal{L} = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t})$$

this is exactly **token-level cross-entropy** — the same loss as classification

Every position is a training example

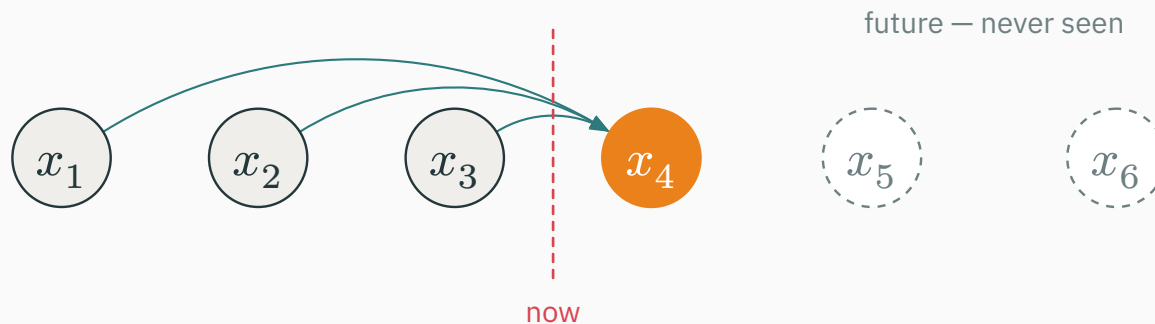
One name yields **many** supervised (context $\rightarrow$ target) pairs. For emma:

context $x_{<t}$	target x_t
. (start)	e
. e	m
. e m	m
. e m m	a
. e m m a	. (end)

one 4-letter name $\rightarrow$ 5 training pairs. A corpus gives **millions**.

Causal / autoregressive v

Predicting x_t may look at the **past** $x_{<t}$ — never at x_t or the future:



autoregressive: each token is predicted from the tokens before it

Tokens and vocabulary

Text must become integers

A neural network consumes **numbers**, not strings. So we fix a **vocabulary** and map each token to an integer **id**:

$\text{cat} \rightarrow 17, \quad \text{dog} \rightarrow 18, \quad . \rightarrow 0$

- the vocabulary V is the fixed set of tokens the model knows;
- a sequence of text becomes a sequence of **token ids**.

makemore: $V = \{ ., a, b, \dots, z \} - 27$ symbols

What counts as a token?

The **granularity** of tokenization is a design choice:

Unit	Vocabulary size	Sequence length
character	tiny (10^2)	very long
word	huge (10^5+)	short
subword	moderate (10^4)	moderate

Subword tokenization (e.g. **BPE**) is the modern default — a middle ground. We won't deep-dive it here; makemore just uses **characters**.

Special tokens

Beyond ordinary symbols, we reserve a few **control** tokens:

Token	Role
<BOS> / .	marks the start of a sequence
<EOS> / .	marks the end — lets the model stop
<PAD>	fills short sequences to a common length
<UNK>	stands in for out-of-vocabulary tokens

makemore reuses a single . as both start and end marker

A token id is not a number

The id is a **name**, not a **quantity**:

cat $\rightarrow$ 17, dog $\rightarrow$ 18 **but** dog - cat $\neq$ 1

Ids are **categorical** indices. Token 18 is not “one more than” token 17 — the integers carry **no** magnitude or order. Feeding the raw id as a scalar feature would invent a meaningless numeric scale.

we need a representation with **no spurious arithmetic — next**

One-hot vectors and embeddings

One-hot: the honest encoding

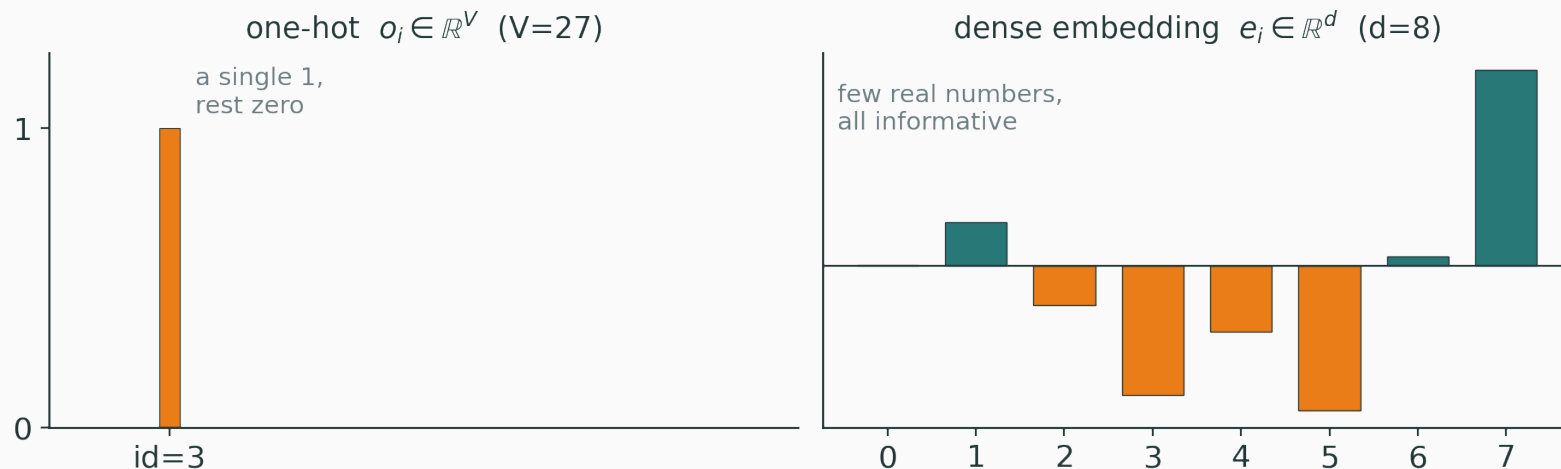
Represent token i as a vector that is **1** in position i , **0** elsewhere:

$$o_i \in \mathbb{R}^V, \quad (o_i)_j = \begin{cases} 1 & j = i \\ 0 & \text{otherwise} \end{cases}$$

- no false ordering — every token is a distinct axis;
- but **high-dimensional** (V can be 50,000) and **sparse**;
- and **no notion of similarity** — every pair is equally far apart.

$$\|o_i - o_j\|^2 = 2 \text{ for } \textbf{every} \text{ distinct pair} - \text{“cat” is as close to “dog” as to “the”}$$

One-hot vs dense embedding v



replace a long sparse indicator with a short **dense** vector of learned features

Embeddings: a learned dense vector

Give each token a **dense** vector $e_i \in \mathbb{R}^d$ with $d \ll V$. Stack them as rows of a matrix:

$$E \in \mathbb{R}^{V \times d}, \quad e_i = E_{i,:} \quad (\text{the } i\text{-th row})$$

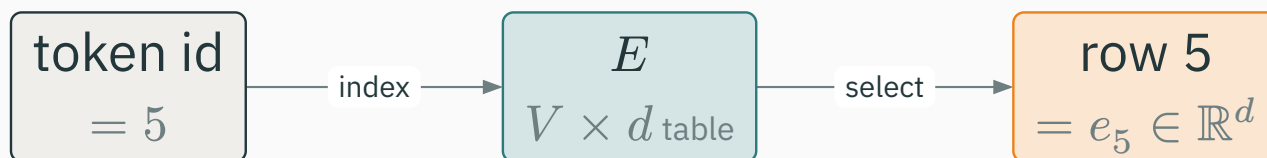
- $V = 27, d = 2$ — small enough to **plot** (makemore);
- $V = 50,000, d = 256$ — a realistic language model.

from a V -dim sparse axis to a d -dim dense point — d is the **embedding dimension**

Lookup is a matrix multiply **D**

Selecting row i of E **is** a linear operation on the one-hot vector:

$$e_i = E^\top o_i$$



in code it is a **table lookup** — but mathematically it is linear, so gradients flow through it

Worked example: embedding lookup

Four tokens [I, like, cats, dogs], embedding dimension $d = 2$:

$$E = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ -1 & 1 \end{pmatrix}, \quad e_I = (1 \ 0), \quad e_{\text{like}} = (0 \ 1)$$

Context "I like" looks up two rows:

$$(e_I, e_{\text{like}}) = ((1, 0), (0, 1))$$

two integer ids in $\rightarrow$ two dense rows out — this is the whole embedding layer

The embedding table E **v**

E is just a matrix of parameters; a context selects a few **rows**:

$E \in \mathbb{R}^{V \times d}$ (rows = tokens, cols = features)

token	.	+0.3	+0.8	+0.3	-1.3	
	e	+0.9	+0.4	-0.5	+0.6	$\leftarrow e_e$
	m	+0.4	+0.3	+0.0	+0.5	
	a	-0.7	-0.2	-0.5	+0.6	
	o	+0.0	-0.3	-0.8	-0.3	$\leftarrow e_o$
	l	+0.0	-0.3	+1.3	+1.0	
	i	-2.7	-1.9	-0.2	-0.4	
	v	+0.2	+0.2	+2.1	-1.1	

looking up **e** and **o** picks out two rows of E — nothing else is touched

Embeddings are parameters

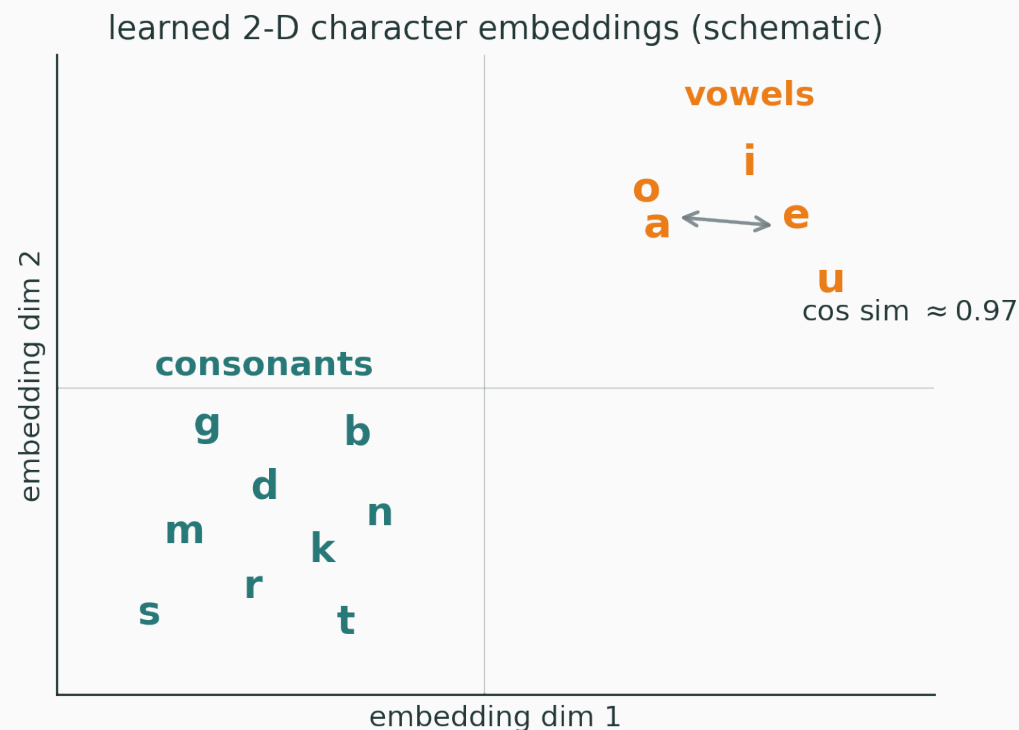
E starts as **small random noise** and is **learned** by gradient descent, like any weight:

$$E \leftarrow E - \eta \nabla_E \mathcal{L}$$

Only the **rows that were looked up** in a batch receive a gradient — an unseen token's embedding does not move on that step. Frequent tokens get trained the most.

Embedding geometry v

Trained embeddings place **similar-context** tokens **near** each other:



in makemore, vowels and consonants tend to cluster — measured by **cosine similarity**

A caveat on embedding geometry

- similarity emerges from **usage** — tokens used in like contexts drift together;
- cosine similarity $\cos(e_i, e_j) = \frac{e_i^\top e_j}{\|e_i\| \|e_j\|}$ measures it;

Do **not** over-promise. Not every direction is a clean semantic axis, and small models learn only coarse structure. “Vowels cluster” is a **tendency**, not a law.

Interactive: embedding lookup

INTERACTIVE

(to build) — click a token id and watch the corresponding **row** of E light up and drop out as the dense embedding vector; nudge a value and see the point move in 2-D.

A fixed-context MLP language model

The full history is too much

Exactly, $p(x_t \mid x_{<t})$ conditions on **all** previous tokens — an unbounded input.
Simplify: condition on only the previous k tokens (a **fixed context window**):

$$p_{\theta}(x_t \mid x_{<t}) \approx p_{\theta}(x_t \mid x_{t-k:t-1})$$

a bounded, fixed-size input $\rightarrow$ an ordinary MLP can handle it. This is the Bengio-2003 / makemore model.

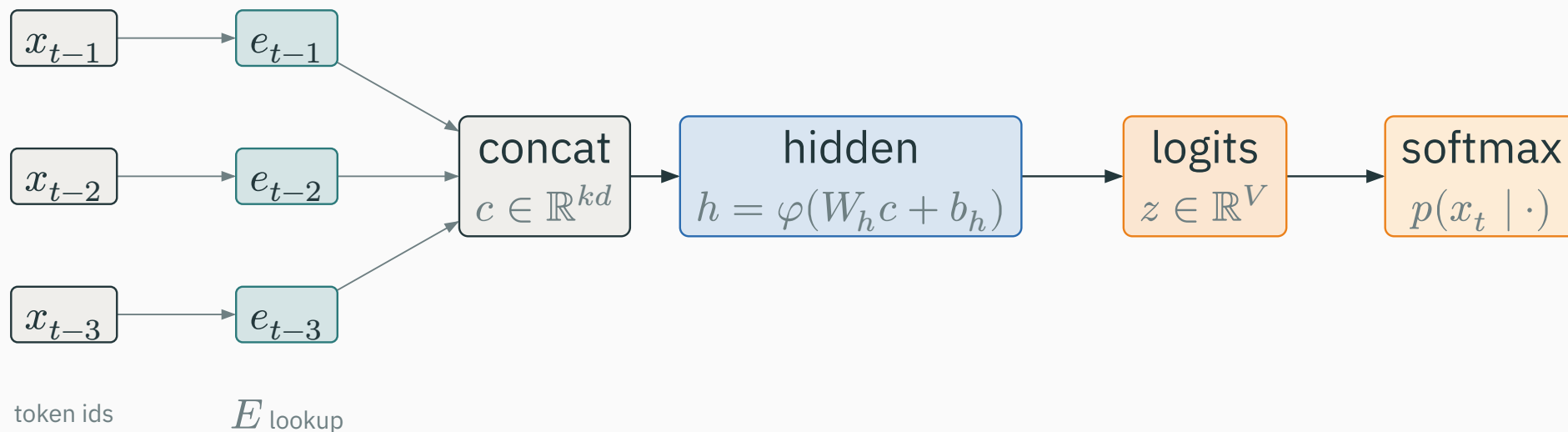
The sliding-window dataset

Slide a window of length k over each name to make (context $\rightarrow$ target) pairs. emma, $k = 3$:

context ($k = 3$)	target
. . .	e
. . e	m
. e m	m
e m m	a
m m a	. (end)

pad the start with .; every window is one supervised example

The MLP language model v



look up $\rightarrow$ concatenate $\rightarrow$ hidden layer $\rightarrow$ logits $\rightarrow$ softmax over V

The model, in equations

Look up each context token's embedding, then run an MLP:

$$e_{t-j} = E[x_{t-j}], \quad c_t = [e_{t-k}; \dots; e_{t-1}] \in \mathbb{R}^{kd}$$

$$h = \varphi(W_h c_t + b_h), \quad z = W_o h + b_o$$

$$p = \text{softmax}(z) \in \mathbb{R}^V$$

Bengio et al. 2003 · Karpathy's makemore — embeddings + one hidden layer + softmax

Why concatenate, not average?

Concatenation **keeps position information**:

$$[e_{\text{dog}}; e_{\text{bites}}; e_{\text{man}}] \neq [e_{\text{man}}; e_{\text{bites}}; e_{\text{dog}}]$$

Averaging the embeddings would give $\frac{1}{k} \sum_j e_{t-j}$ — identical for any **reordering** of the context. That destroys order, the one thing a sequence model must keep.

each position lands in its **own** slice of c_t , hitting its **own** columns of W_h

Tensor shapes

Track a batch of B contexts through the model:

Tensor	Shape	Meaning
context ids X	$B \times k$	k token ids per row
embeddings $E[X]$	$B \times k \times d$	one d -vector per token
concat C	$B \times kd$	flatten the k embeddings
hidden H	$B \times H_{\text{dim}}$	after W_h and φ
logits Z	$B \times V$	one score per vocab token

Parameter count D

Three weight blocks — embedding, hidden, output:

$$\underbrace{Vd}_{\text{embedding}} + \underbrace{H \cdot kd + H}_{\text{hidden}} + \underbrace{V \cdot H + V}_{\text{output}}$$

where H is the hidden width and k the context length.

note: the **embedding** and **output** blocks both scale with the vocabulary V

Worked parameter count D

Take $V = 20,000$, $d = 128$, $k = 4$, $H = 512$:

$$\text{embedding} = Vd = 20,000 \cdot 128 = 2,560,000$$

$$\text{hidden} = H \cdot kd + H = 512 \cdot 512 + 512 = 262,656$$

$$\text{output} = VH + V = 20,000 \cdot 512 + 20,000 = 10,260,000$$

total $\approx$ 13.08 M parameters

embedding + output (both $\propto V$) dominate — the hidden layer is tiny by comparison

A fully-worked forward and backward pass

The tiny setup

Vocabulary [I, like, cats, dogs], context "I like", target cats, $d = 2$:

$$e_I = (1 \ 0), \quad e_{\text{like}} = (0 \ 1), \quad c = (1 \ 0 \ 0 \ 1)^\top$$

$$W_h = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}, \quad b_h = 0$$

small enough to do **entirely by hand** — and check against the earlier formulas

Forward: the hidden layer

Multiply and apply ReLU:

$$W_h c = (1 \cdot 1 + 0 \cdot 0 + 0 \cdot 0 + 1 \cdot 1 \quad 0 + 0 + 0 + 0) = (2 \quad 0)$$

$$h = \text{ReLU}(W_h c) = (2 \quad 0)$$

the first hidden unit fires; the second is switched off

Forward: logits and softmax

Suppose the output layer produces logits (for [I, like, cats, dogs]):

$$z = (-2 \ 0 \ 2 \ 1)$$

Softmax needs the normalizer:

$$Z = e^{-2} + e^0 + e^2 + e^1 \approx 0.135 + 1 + 7.389 + 2.718 = 11.243$$

$$p = (0.012 \ 0.089 \ 0.657 \ 0.242)$$

the model already puts most mass (0.657) on the correct token `cats`

Forward: the loss D

Cross-entropy picks out the **target** probability. Target is cats:

$$\mathcal{L} = -\log p_{\text{cats}} = -\log(0.657) \approx 0.42$$

$$\mathcal{L} \approx 0.42 \text{ nats}$$

a confident-but-imperfect prediction; a perfect one would give $\mathcal{L} = 0$

Backward: the output gradient

For softmax + cross-entropy, the gradient w.r.t. the logits is beautifully simple:

$$\nabla_z \mathcal{L} = p - y, \quad y = (0 \ 0 \ 1 \ 0) \quad (\text{one-hot target})$$

$$\nabla_z \mathcal{L} = (0.012 \ 0.089 \ -0.343 \ 0.242)$$

push down the three wrong logits, **push up** the correct one `cats` — that is all learning is here

Backward: down to the embeddings

The chain rule walks the gradient back through the MLP:

$$\nabla_h \mathcal{L} = W_o^\top \nabla_z \mathcal{L}, \quad \nabla_c \mathcal{L} = W_h^\top (\nabla_h \mathcal{L} \odot \text{ReLU}'(W_h c))$$

With our numbers this gives

$$\nabla_c \mathcal{L} = (-0.234 \ 0 \ 0 \ -0.234)$$

Split $c = [e_{\text{I}}; e_{\text{like}}]$ back into the two rows that were looked up:

$$\nabla_{e_{\text{I}}} \mathcal{L} = (-0.234 \ 0), \quad \nabla_{e_{\text{like}}} \mathcal{L} = (0 \ -0.234)$$

only rows `I` and `like` of E get updated this step — the tokens the context actually used

Interactive: the MLP language model I

INTERACTIVE

(to build) — step through this exact example: watch the embeddings look up, the concat form, the softmax bar chart appear, and the $p - y$ gradient flow back and nudge the two used rows.

Training objective and evaluation

The dataset loss

Average the per-token NLL over every (context $\rightarrow$ target) pair in the corpus:

$$\mathcal{L} = -\frac{1}{T-k} \sum_t \log p_\theta(x_t \mid x_{t-k:t-1})$$

the token-level cross-entropy from Part II — now over the whole sliding-window dataset

The minibatch loop

Train by the same loop as any classifier — on a minibatch B of contexts:

$$\mathcal{L}_B = -\frac{1}{|B|} \sum_{i \in B} \log p_{\theta}(y_i \mid c_i)$$



Padding and loss masks

Sequences in a batch have **different lengths**. Pad short ones to a common length:

- introduce a <PAD> token so every row is length k ;
- but **do not** let padding contribute to the loss.

Use a mask $m_{it} \in \{0, 1\}$ (1 = real token, 0 = pad):

$$\mathcal{L} = \frac{\sum_{i,t} m_{it} (-\log p_{it})}{\sum_{i,t} m_{it}}$$

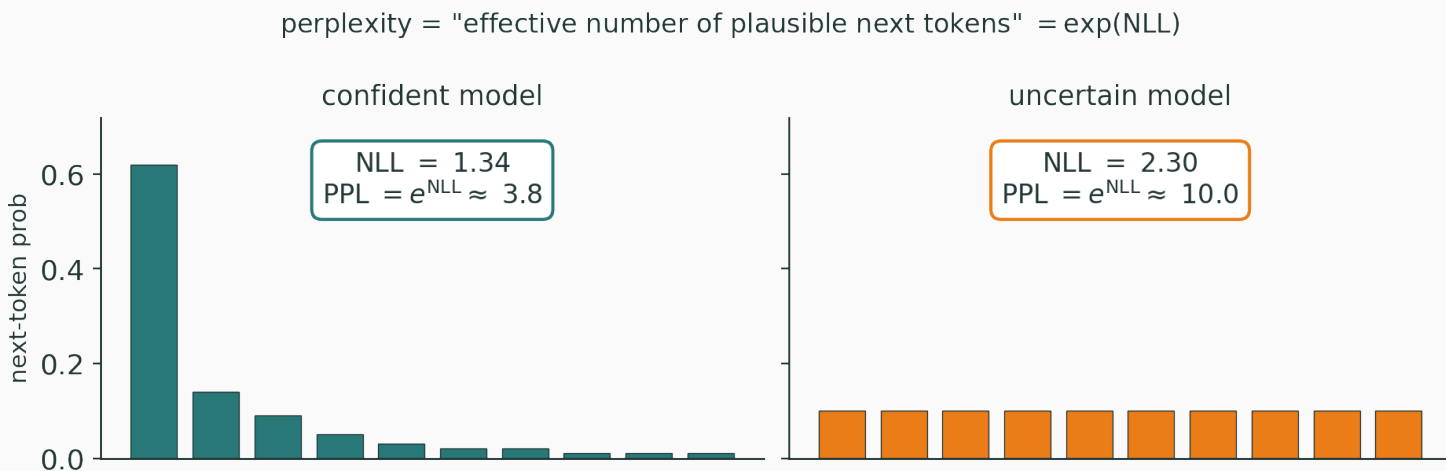
the mask makes padding **invisible** to the gradient — only real tokens count

Report the **per-token** NLL exponentiated — the **perplexity**:

$$\text{PPL} = \exp(\text{NLL}) = \exp\left(-\frac{1}{N} \sum_t \log p_\theta(x_t \mid x_{<t})\right)$$

Interpretation: the **effective number of plausible next tokens**. $\text{PPL} = 1$ means perfectly certain; $\text{PPL} = V$ means no better than a uniform guess.

Perplexity, two ways v



a peaked distribution has **low** perplexity; a flat one is **perplexed** among many tokens

Perplexity: worked and cautioned

Quick sanity checks (natural log):

$$\text{NLL} = \log 2 \approx 0.69 \quad \Rightarrow \quad \text{PPL} = e^{\log 2} = 2$$

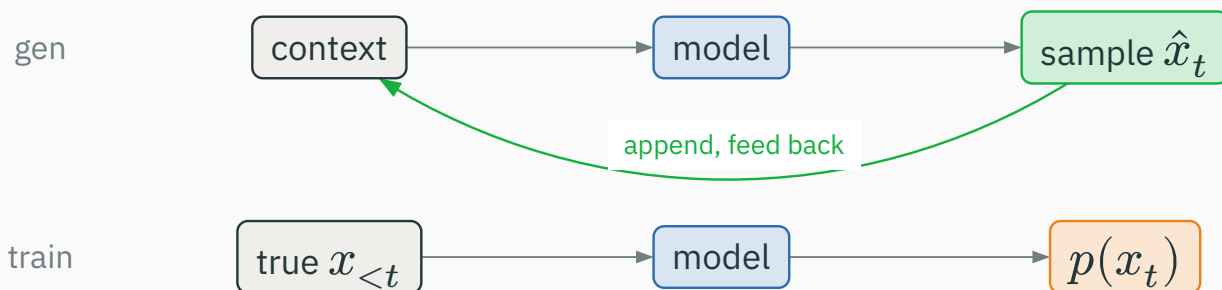
$$\text{NLL} = \log 10 \approx 2.30 \quad \Rightarrow \quad \text{PPL} = e^{\log 10} = 10$$

Caveat. Perplexity depends on the **tokenization**. A character model and a word model are **not** comparable by PPL — they predict different-sized units.

Autoregressive generation

Training vs generation v

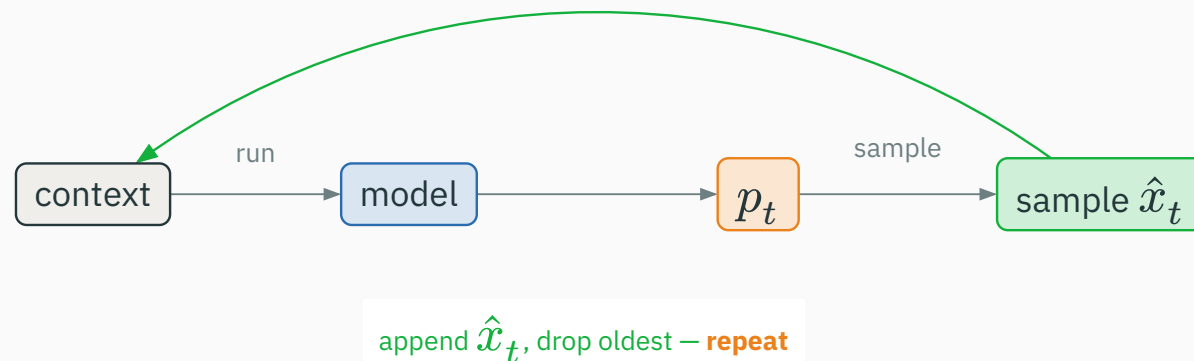
At **training** time the previous tokens are **true**; at **generation** time they are the model's **own** outputs, fed back in:



The generation algorithm

One token at a time:

1. start from a context (e.g. just . for makemore);
2. run the model $\rightarrow$ next-token distribution p_t ;
3. **select** or **sample** a token $\hat{x}_t$ from p_t ;
4. **append** it; drop the oldest token if the context is fixed-length;
5. repeat until $\langle \text{EOS} \rangle$ or a maximum length.



Greedy decoding

Always take the **most probable** token:

$$\hat{x}_{t+1} = \arg \max_v p_t(v)$$

Pro — deterministic, simple, always picks the model's best guess.

Con — repetitive and bland; locally-best $\neq$ globally-best sequence.

Sampling

Draw the next token **at random** from the distribution:

$$\hat{x}_{t+1} \sim \text{Categorical}(p_t)$$

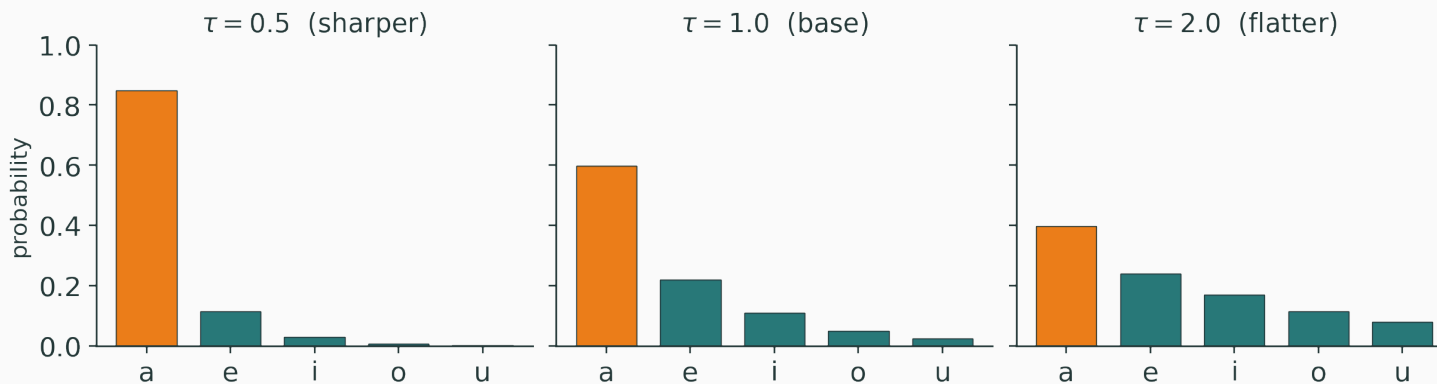
- respects the model's uncertainty — likely tokens are likely, but not forced;
- produces **diverse** outputs across runs;
- makemore sampling character-by-character gives **new** plausible names.

. -> e -> m -> m -> a -> . yields emma; a different draw yields elian, amara, ...

Rescale the logits by τ before the softmax:

$$p_i(\tau) = \text{softmax}(z/\tau)_i$$

same logits $z = [2, 1, 0.3, -0.5, -1.2]$, $\text{softmax}(z/\tau)$



$\tau < 1$ sharpens (bolder) · $\tau > 1$ flattens (more random) · $\tau \rightarrow 0 \rightarrow$ greedy

Top-k sampling

Keep only the k most probable tokens, **renormalize**, then sample:

- zero out everything outside the top k ;
- rescale the survivors to sum to 1;
- prevents rare, low-probability tokens from ever being drawn.

nucleus / top- p sampling is the close cousin — we meet it in the Transformer lecture

Interactive: next-token sampling I

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/softmax-temperature

Sweep temperature and top- k on a fixed set of logits and watch the next-token bar chart sharpen or flatten — then sample repeatedly and see the diversity of outputs change.

Where the fixed-context MLP fails

It only sees a fixed window

The MLP conditions on exactly the last k tokens — anything before x_{t-k} is **invisible**:

“The book that I placed on the table near the window ... **was**”

To choose **was** vs **were** the model needs the subject **book** — but with a small window that word has already scrolled out of view. Long-range dependencies are lost.

Longer context costs parameters

Widening the window grows the hidden weight matrix:

$$W_h \in \mathbb{R}^{H \times kd} \Rightarrow H \cdot kd \text{ parameters}$$

- double the context $k \rightarrow$ double the hidden-layer parameters;
- and the model **still** cannot accept a sequence longer than k .

you cannot simply set $k = \infty$ — cost grows with the window, and k is fixed at build time

Position-specific parameters

Each context slot hits a **different** block of columns of W_h :

$$c_t = [e_{t-k}; \dots; e_{t-1}], \quad \text{slot } j \rightarrow \text{its own } d\text{-column block of } W_h$$

The **same** token at position 1 vs position 2 is processed by **different** weights. Order is preserved, but computation is **not shared** across time — the model relearns each position separately.

What we actually want

A better sequence model should:

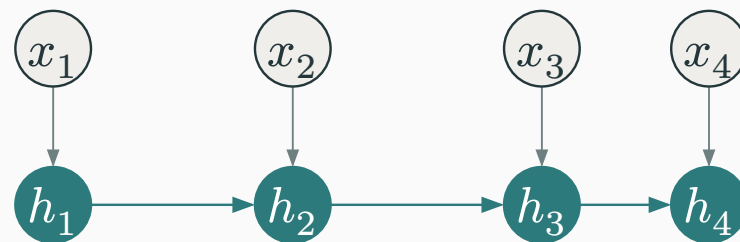
- accept **variable-length** input (no fixed k);
- **share** parameters across time steps;
- **carry** information from arbitrarily far back;
- emit an output at **every** step.

a recurrent update: $h_t = f_{\theta}(h_{t-1}, x_t)$

From fixed window to recurrent state **v**



fixed window (MLP)



state carries the past

recurrent (RNN)

The recurrent idea

Replace the fixed window with a **running state** that summarizes the past:

$$\underbrace{p(x_{t+1} \mid x_{t-k:t})}_{\text{MLP: fixed window}} \longrightarrow \underbrace{h_t = f_{\theta}(h_{t-1}, e_t)}_{\text{RNN: recurrent state}}$$

$$p = \text{softmax}(Wh_t + b)$$

The hidden state h_t is a **learned summary** of everything seen so far — one shared function f_{θ} applied at every step, for any length.

One-slide summary

Idea	Takeaway
chain rule	$p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$ — model every conditional
language modeling	repeated next-token classification; loss = token cross-entropy
tokens	text $\rightarrow$ integer ids; ids are categorical, not numeric
embeddings	learned dense rows of E ; similar tokens land nearby
MLP-LM	fixed window $\rightarrow$ concat $\rightarrow$ MLP $\rightarrow$ softmax (Bengio / makemore)
generation	autoregressive; greedy / sample / temperature / top- k
limitation	fixed window, position-specific weights $\rightarrow$ motivates RNNs

Final mental model — one idea

A language model estimates a **distribution over the next token.**

Language modeling is **repeated classification**

tokens → embeddings → context → softmax over the vocabulary

We predicted the next token from a **fixed window**.

Next: **Sequence Models — RNNs** — hidden state, backprop through time, and LSTMs / GRUs that carry information across a whole sequence.